

1章 イントロダクション

1.1 ソフトウェアアーキテクチャの定義

ソフトウェアアーキテクチャの構成要素

ソフトウェアアーキテクチャは、以下の4要素から構成される

要素	定義	ポイント	例
構造	システムを構成する部品とその関係	コンポーネントやモジュール、それらの接続・依存関係	レイヤード、マイクロサービスなど
アーキテクチャ特性 (→ 4章)	システムに求められる、ドメインに依らない設計上の考慮事項	1. ドメインに依らない、設計に関する考慮事項を明らかにするもの (≒ 非機能要件) 2. 設計の構造的な側面に影響を与えるもの 3. アプリケーションの成功に不可欠な重要なもの	セキュリティ、デプロイ容易性など
アーキテクチャ決定 (→ 19章)	構造を守るための制約・ルール	「～してはならない」「～を使うこと」のように、設計の選択肢を制限する	「ビジネス層とサービス層だけが永続化層にアクセスできる」
設計指針	設計を導くガイドライン	アーキテクチャ決定より柔軟で、状況に応じて例外や選択肢を許容する	「サービス間の通信に非同期通信を用いることでパフォーマンスを向上すべき」

1.4 ソフトウェアアーキテクチャの法則

著者らの経験による普遍的な原理として、次の二つの法則が紹介されている:

第一法則: 「ソフトウェアアーキテクチャはトレードオフがすべてだ」

- 必然的帰結その一: 「アーキテクトが、トレードオフでない何かを見出したと考えているなら、まだトレードオフを特定していないだけの可能性が高い」
- ベストなソリューションが一つに決まるわけではない。トレードオフのある選択肢の中から、ビジネス上の要件を踏まえた上で、ベター (または、最悪でない) ものを選択するという姿勢が大事になる

第二法則: 「『どうやって』よりも『なぜ』の方がずっと重要だ」

- アーキテクチャとはトレードオフなのだから、意思決定をするということは、その時点で見えている範囲でのConsを受け入れるという選択肢をする、ということ
- しかし、今あるもの (ソフトウェア) から、なぜその選択をしたのかを知るのはほとんど不可能である
- よって、決断した時の状況を記録しておくことが重要である (→ ADR (Architecture Decision Record) (19章))

参考文献

- [アーキテクチャ特性について](#)
- [ARDについて](#)